

[Open in app](#)[Sign up](#)[Sign in](#)**Medium** Search**Geek Culture** · [Follow publication](#)

Semantic Segmentation with SegFormer

6 min read · Jan 6, 2022



Praneet Bomma

[Follow](#)

Listen



Share



Model Prediction on Drone Dataset Image

Introduction

Image Segmentation is the process of classifying each pixel in an image. It is a computer vision task tasked mainly to detect regions in an image with an object.

Today we will be covering Semantic Segmentation using SegFormer. It is a neural network architecture inspired by an architecture that is the talk of the town nowadays, Transformers. You must be living under the rock if that doesn't ring any bells!

The Segformer architecture doesn't depend on positional encodings like Vision Transformers and hence improves inference on images of different resolutions. The other thing that ranks Segformer above the rest of its counterparts is the way its Decoder has been designed. Unlike all the Segmentation architectures that use Upsampling or Deconvolutions, Segformer uses MLP Decoder which is faster and more efficient. We will be discussing the architecture in a bit more detail in the following sections.

Transformers

What are these Transformers really?

Transformers were originally built to solve the Sequence2Sequence problems like text generation and translation. Transformer is a novel architecture for transforming one sequence into another using an Encoder and Decoder along with the self-attention mechanism.

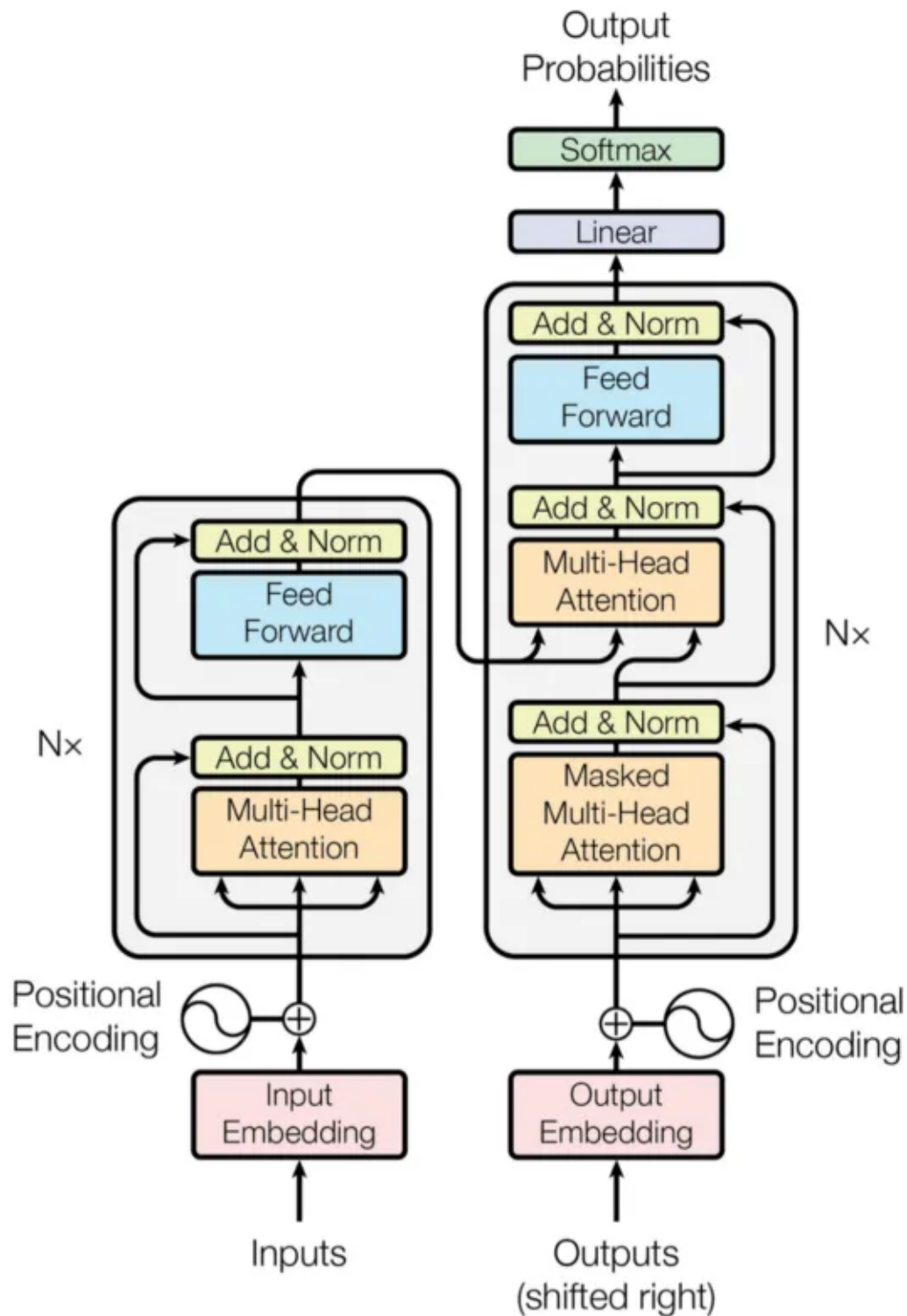


Figure 1: The Transformer - model architecture.

Figure 1: From 'Attention Is All You Need' by Vaswani et al.

SegFormer Architecture

The architecture is based on a Transformer architecture with Encoder-Decoder heads where the encoder makes use of Self Attention.

The encoder architecture is hierarchical in nature which outputs multiscale features. It does not need positional encoding, thereby avoiding the interpolation of positional codes which leads to decreased performance when the testing resolution differs from training.

The architecture unlike other complex decoders applies a simple MLP decoder that aggregates information from different layers and thus combines both local attention and global attention for rendering powerful representations.

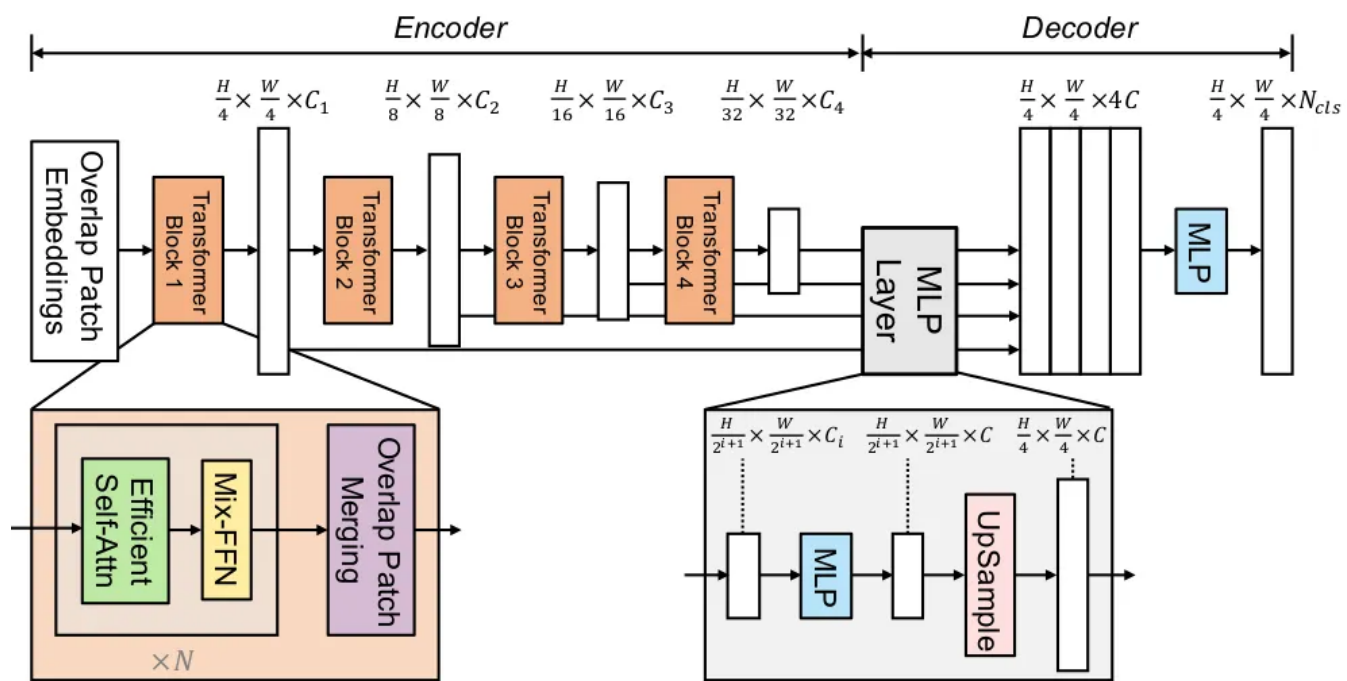


Figure 2: SegFormer architecture

Semantic Segmentation

For this blog, we will be training a semantic segmentation model with SegFormer on Drone Dataset which can be downloaded from Kaggle.

Dataset Overview

The Semantic Drone Dataset focuses on semantic understanding of urban scenes for increasing the safety of autonomous drone flight and landing procedures. The imagery depicts more than 20 houses from nadir (bird's eye) view acquired at an

altitude of 5 to 30 meters above the ground. A high-resolution camera was used to acquire images at a size of 6000x4000px (24Mpx). The training set contains 400 publicly available images.

Data Augmentation

As the images are of size 6000x4000px which is too big to train our model on, we will apply a cropping augmentation step to pick crops of 2000x2000px with a sliding window mechanism. This gives us 6 crops per image which increases the dataset size to 2400 images. We will be using 2100 images for training and 300 images for validation. The augmented dataset can be downloaded from [Google Drive](#).

Training

Training the SegFormer model is as easy as training any other model in PyTorch. The steps we will be following will be:

- Loading the data with Dataloader.
- Initializing hyperparameters and the optimizer.
- Writing the training loop

We will be using HuggingFace's feature extractor which takes care of handling the segmentation labels directly from the loaded mask.

```
encoded_inputs = self.feature_extractor(augmented['image'],
augmented['mask'], return_tensors="pt")
```

We have the labels CSV which can be used for plotting the colors on the segmented image.

```
classes = pd.read_csv('class_dict_seg.csv')['name']
id2label = classes.to_dict()
label2id = {v: k for k, v in id2label.items()}
```

We will be using HuggingFace's pre-trained SegFormer model and fine-tune it with our own dataset. The `transformers` library by HuggingFace makes it really easy to use a pre-trained model for fine-tuning it with a custom dataset. Loading the pre-trained model is barely a couple of lines.

```
feature_extractor = SegformerFeatureExtractor(align=False,
reduce_zero_label=False)

model = SegformerForSemanticSegmentation.from_pretrained(
    "nvidia/mit-b5", ignore_mismatched_sizes=True,
    num_labels=len(id2label), id2label=id2label, label2id=label2id,
    reshape_last_stage=True)
```

We will be using the default hyperparameters used in training the SegFormer for training the model with the Drone Dataset. The `transformers` library by HuggingFace provides the optimizer `AdamW` with slight changes to handle training the pre-trained HuggingFace model.

```
optimizer = AdamW(model.parameters(), lr=0.00006)
```

And here we are ready to train the model. We are training it only for 10 epochs for the sake of the blog as it takes a long time on Colab to train and gives decent results with 10 epochs.

Inference

The above model trained on Drone Dataset is pushed to the HuggingFace Model hub.

Let's look at the inference code to test our model.

Get Praneet Bomma's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐

Remember me for faster sign in

Getting the colors for the color palette that we will be using to draw the classified segmentation on the image.

```
df = pd.read_csv('drone_dataset/class_dict_seg.csv')
classes = df['name']
palette = df[[' r', ' g', ' b']].values
id2label = classes.to_dict()
label2id = {v: k for k, v in id2label.items()}
```

As we saw earlier, loading the pre-trained model is very easy with HuggingFace.

Note: We will be using our pre-trained model instead of Nvidia's pre-trained model. The model is pushed into the repository named `deep-learning-analytics/segformer_semantic_segmentation` in HuggingFace Model Hub.

```
feature_extractor = SegformerFeatureExtractor(aligned=False,
reduce_zero_label=False)

device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")

model = SegformerForSemanticSegmentation.from_pretrained("deep-
```



```
learning-analytics/segformer_semantic_segmentation",
ignore_mismatched_sizes=True,

num_labels=len(id2label), id2label=id2label, label2id=label2id,
reshape_last_stage=True)

model = model.to(device)
```

The only thing we need to do to get it ready for the model inference is to pass the loaded image through the `feature_extractor` that we talked about earlier.

```
# prepare the image for the model (aligned resize)

feature_extractor_inference =
SegformerFeatureExtractor(do_random_crop=False, do_pad=False)

pixel_values = feature_extractor_inference(image,
return_tensors="pt").pixel_values.to(device)

outputs = model(pixel_values=pixel_values) # logits are of shape
(batch_size, num_labels, height/4, width/4)

logits = outputs.logits.cpu()
```

Results



Model Prediction Examples on Drone Dataset Images

And that's it. We did it! Pat your back, you trained a transformer model all on your own!

You can find the code for training and inference on [Google Colab](#) or on [Github](#)

You can also try out the model with your own data with the model deployed on [HuggingFace Spaces](#).

Conclusion

We implemented a Semantic Segmentation model based on Transformer which has given state-of-the-art results in multiple tasks. We also saw how the architecture was designed taking into consideration each aspect of the problem. The hierarchical design helps propagate features at multiple scales. The MLP decoder helps speed up the forward pass which significantly improves the FPS of the model. We also tried HuggingFace which makes it so very easy to train and try Transformer based

models.

I hope you take something away from the blog. Please do try out the colab notebook and share your experiences in the comments below.

At [Deep Learning Analytics](#), we are extremely passionate about using Machine Learning to solve real-world problems. We have helped many businesses deploy innovative AI-based solutions. Contact us through our website [here](#) if you see an opportunity to collaborate.

Important Links

- [Google Colab](#)
- [Original Drone Dataset](#)
- [Augmented Drone Dataset](#)
- [Github](#)
- [HuggingFace Model Hub](#)
- [Hugging Face Spaces](#)

References

GitHub - NVlabs/SegFormer: Official PyTorch implementation of SegFormer

Figure 1: Performance of SegFormer-B0 to SegFormer-B5.
SegFormer: Simple and Efficient Design for Semantic...

github.com

SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers

We present SegFormer, a simple, efficient yet powerful semantic segmentation framework which unifies Transformers with...

arxiv.org

Attention Is All You Need

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an...

arxiv.org

Hugging Face - The AI community building the future.

We're on a journey to advance and democratize artificial intelligence through open source and open science.

huggingface.co

ICG - DroneDataset

The Drone Dataset is made freely available to academic and non-academic entities for non-commercial purposes such as...

www.tugraz.at

Transformers

Computer Vision

Deep Learning

Semantic Segmentation

Artificial Intelligence

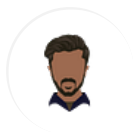


Follow

Published in Geek Culture

34K followers · Last published Aug 31, 2023

A new tech publication by Start it up (<https://medium.com/swlh>).



Follow

Written by Praneet Bomma

82 followers · 23 following

Machine Learning Engineer

